

# Regularization

## A Beginner's Companion Guide

---

*How to stop a network from memorizing — L2, dropout, and the parts that make them work*

---



# About This Guide

## Teaching a Powerful Network to Stop Memorizing and Start Generalizing

*A plain-language walkthrough of L2 regularization and dropout — and the questions they raise*

Deep networks are powerful enough to be dangerous to themselves. Give one enough capacity and it will happily memorize every quirk and stray data point in your training set, scoring beautifully there while failing on anything new. That failure has a name — overfitting — and the family of cures has a name too: **regularization**. In this assignment you took a network that overfits a noisy dataset and applied two of the most effective regularization techniques in deep learning, watching the test accuracy climb each time.

The setting is a small, playful one: you are advising the French national football team on where their goalkeeper should kick the ball so a teammate can head it. You have a 2D dataset of past plays — blue dots where France won the header, red dots where the opponent did — and your job is to find the boundary between them. A diagonal line roughly separates the two, but the data is noisy, and a powerful network is tempted to wiggle around every single point. This guide walks through the code you wrote, defines each concept in plain terms, and answers the questions the assignment raises along the way.

### HOW TO READ THIS GUIDE

Each section follows the same rhythm: the concept in everyday language, the code that implements it, and a short note on where the idea lives in the wider world of deep learning. You need no mathematics beyond multiplication, squaring, and the idea of an average. The five questions posed in the assignment are answered directly in the sections where they naturally arise, and gathered again at the end.

Before diving in, here are the five questions this guide sets out to answer, so you can watch for them: what regularization is and what a *baseline model* means; what a *decision boundary* is; why we bother *computing costs*; why *backpropagation* must change when you regularize; and how to tell whether a model is *overfitting or underfitting*. Each is answered in plain terms below.

## 1. What Regularization Is, and the Baseline Model

Start with the problem. You first trained the network with no regularization at all, letting it do whatever it liked with the data. It reached about **95% accuracy on the training set but only 91.5% on the test set**. That gap — great on data it has seen, weaker on data it has not — is the fingerprint of overfitting. Plotting its decision boundary shows a wiggly, contorted line bending around individual noisy dots

rather than following the clean diagonal trend. This first, unregularized network is your **baseline model**.

#### KEY CONCEPT: THE BASELINE MODEL

A baseline model is the plain, first-attempt version you build before adding any improvements — here, the network with no regularization. Its purpose is to give you a reference point. Every later technique is judged by how much it improves on the baseline. Without a baseline you cannot tell whether a change actually helped, so establishing one is always the sensible first step.

So what is regularization? It is any technique that deliberately constrains a model — makes its job a little harder — so that it cannot overfit so easily. It trades a small amount of training-set performance for a meaningful gain on unseen data. As the lecture notes put it: if you suspect your network is overfitting (a high-variance problem), regularization is one of the very first things to try. The other reliable cure is simply gathering more data, but data is often expensive or impossible to get more of, whereas regularization costs you nothing but a few lines of code.

#### KEY CONCEPT: REGULARIZATION

Regularization is a set of techniques that reduce overfitting by discouraging a model from becoming too complex. Rather than letting the network fit every training point exactly, regularization gently pushes it toward simpler, smoother solutions that generalize better to new data. The two techniques in this assignment — **L2 regularization** and **dropout** — are among the most widely used in all of deep learning.

## 2. The Decision Boundary

Throughout the assignment you plotted something called the decision boundary after each model. It is the single most useful picture for understanding what a classifier has learned, so it is worth defining carefully.

#### KEY CONCEPT: THE DECISION BOUNDARY

A decision boundary is the line (or curve) a classifier draws to separate one class from another. On the football field, it is the dividing line where the model switches from predicting "blue wins the header" to "red wins it." On one side it predicts blue; on the other, red. The *shape* of that boundary reveals everything about the model's health: a smooth, simple boundary suggests the model has learned the real trend, while a jagged, contorted boundary that snakes around individual dots is the visible signature of overfitting.

This is exactly what you saw. The baseline model's boundary was a wiggly mess, twisting to capture noisy outliers. After L2 regularization, the boundary became noticeably **smoother** — it stopped chasing individual points and followed the broad diagonal trend instead. After dropout, it was smoother still. The decision boundary turns the abstract idea of overfitting into something you can simply look at and recognize.

#### WHERE THIS MAPS IN DEEP LEARNING

Foundation: in any two-feature problem you can plot the decision boundary directly and diagnose a model by eye. Advanced: real problems have hundreds or thousands of features, where the boundary lives in a space too high-dimensional to draw. There you fall back on numbers — comparing training error to validation error — to sense the boundary's shape indirectly. The intuition you build here on a 2D picture is what you carry into those invisible high-dimensional cases.

### 3. Why Costs Are Computed

In every model you trained, a **cost** was computed at each iteration. Understanding why is central to understanding how regularization works, because L2 regularization operates by changing the cost itself.

#### KEY CONCEPT: THE COST FUNCTION

The cost (also called the loss) is a single number that measures how wrong the model currently is across all the training examples — the lower the cost, the better the model fits the data. It exists to give training a target: the entire learning process is just the search for the weights that make this number as small as possible. Without a cost to measure against, the network would have no way of knowing whether an adjustment made things better or worse.

For this classification task the baseline cost is the **cross-entropy cost**, which is large when the model's predicted probabilities are confidently wrong and small when they are confidently right. The crucial insight of L2 regularization is that you can *change what the model is trying to minimize*. By adding an extra term to the cost — one that grows whenever the weights grow large — you make large weights "expensive." The network, always chasing a lower cost, is now pushed to keep its weights small as well as to fit the data:

```
def compute_cost_with_regularization(A3, Y, parameters, lambd):
    m = Y.shape[1]
    w1 = parameters["w1"]
    w2 = parameters["w2"]
    w3 = parameters["w3"]

    # the ordinary cross-entropy cost: "how wrong are the predictions?"
```

```

cross_entropy_cost = compute_cost(A3, Y)

# the L2 penalty: sum of every weight squared, scaled by lambda / (2m).
# This term grows when weights grow, so it makes large weights costly.
L2_regularization_cost = (1/m) * (lambda/2) * (np.sum(np.square(W1))
                                                + np.sum(np.square(W2))
                                                + np.sum(np.square(W3)))

cost = cross_entropy_cost + L2_regularization_cost
return cost

```

The new cost is simply the old cross-entropy cost plus a penalty. That penalty squares every weight, adds them all up across the three weight matrices, and scales the total by `lambda / (2m)`. The hyperparameter `lambda` (spelled without the final "a" because `lambda` is a reserved word in Python) controls how strong the pressure is — turn it up and the network is punished harder for large weights.

#### WHAT L2 REGULARIZATION IS ACTUALLY DOING

L2 rests on a simple assumption: a model with small weights is simpler than one with large weights. By penalizing the squared weights, you drive them all toward smaller values — it becomes too costly to keep them large. Smaller weights make a smoother model whose output changes more slowly as the input changes, which is exactly a less wiggly decision boundary. This pressure toward small weights is so characteristic that L2 regularization is also called **weight decay**. Note that the bias terms are left out of the penalty — almost all the parameters live in the weight matrices, so penalizing the handful of biases would make little difference.

## 4. Why Backpropagation Must Change for Regularization

This answers one of the assignment's direct questions: *why is backpropagation used in regularization?* The short answer is that backpropagation is the mechanism that actually applies the regularization. Changing the cost is only half the job; the network has to be told how to respond to that new cost, and that is precisely what backpropagation computes.

#### KEY CONCEPT: BACKPROPAGATION

Backpropagation is the algorithm that figures out, for every weight in the network, which direction and how much to nudge it to reduce the cost. It does this by computing the **gradient** — the slope of the cost with respect to each weight. Training then takes a small step downhill along that slope. In short, the cost says *how wrong we are*; backpropagation says *what to change to be less wrong*.

Here is the key link: because L2 regularization *added a term to the cost*, the gradient of that cost must change too — you cannot change the target without changing the directions that lead to it. Calculus

tells us the derivative of the penalty term  $(\text{lambda}/2m) \times W^2$  is simply  $(\text{lambda}/m) \times W$ . So each weight's gradient gains one extra piece, proportional to the weight itself:

```
def backward_propagation_with_regularization(X, Y, cache, lambda):
    m = X.shape[1]
    (Z1, A1, W1, b1, Z2, A2, W2, b2, Z3, A3, W3, b3) = cache

    dZ3 = A3 - Y
    # each dW now has its normal term PLUS the regularization gradient (lambda/m)*W
    dW3 = 1./m * np.dot(dZ3, A2.T) + (lambda/m) * W3
    db3 = 1./m * np.sum(dZ3, axis=1, keepdims=True)

    dA2 = np.dot(W3.T, dZ3)
    dZ2 = np.multiply(dA2, np.int64(A2 > 0))
    dW2 = 1./m * np.dot(dZ2, A1.T) + (lambda/m) * W2
    db2 = 1./m * np.sum(dZ2, axis=1, keepdims=True)

    dA1 = np.dot(W2.T, dZ2)
    dZ1 = np.multiply(dA1, np.int64(A1 > 0))
    dW1 = 1./m * np.dot(dZ1, X.T) + (lambda/m) * W1
    db1 = 1./m * np.sum(dZ1, axis=1, keepdims=True)

    gradients = {"dZ3": dZ3, "dW3": dW3, "db3": db3, "dA2": dA2,
                  "dZ2": dZ2, "dW2": dW2, "db2": db2, "dA1": dA1,
                  "dZ1": dZ1, "dW1": dW1, "db1": db1}
    return gradients
```

Notice that only the three weight gradients (`dW1`, `dW2`, `dW3`) change — each gains a  $+(\text{lambda}/m) \times W$  term — while the bias gradients are untouched, mirroring the fact that biases were left out of the penalty. That extra term is the steady downward pressure on weight size, applied on every single iteration. Run with `lambda = 0.7`, this model's test accuracy rose from the baseline's 91.5% to **93%**, and its decision boundary smoothed out. Training accuracy dipped slightly to 94% — the expected, healthy sign that the network can no longer overfit.

### THE IMPLICATIONS OF L2 REGULARIZATION

On the **cost**: a regularization term is added. On **backpropagation**: there are extra terms in the gradients with respect to the weight matrices. On the **weights**: they end up smaller, which is why the technique is nicknamed weight decay. These three changes always travel together — change the cost, and backpropagation and the final weights must follow.

## 5. Dropout: Regularization by Randomly Shutting Neurons Off

Dropout is a regularization technique unique to deep learning, and it works on a completely different principle from L2. Instead of shrinking weights, it **randomly switches off some neurons on every training iteration**. Each pass through the network trains a slightly different, smaller version of itself, because a random subset of neurons is temporarily removed.

#### KEY CONCEPT: DROPOUT

Dropout randomly "shuts down" (sets to zero) each neuron in a layer with some probability on every training step. The fraction it *keeps* is called `keep_prob`; a `keep_prob` of 0.86 means each neuron has an 86% chance of staying active and a 14% chance of being dropped that iteration. Because any neuron might vanish at any moment, the network cannot lean too heavily on any single one. It is forced to spread what it learns across many neurons, which makes it more robust and less prone to overfitting.

Forward propagation with dropout adds four steps to each hidden layer. You build a random mask, convert it to ones and zeros, multiply it into the activations to switch some off, then rescale what remains:

```
def forward_propagation_with_dropout(X, parameters, keep_prob = 0.5):
    np.random.seed(1)
    W1, b1 = parameters["W1"], parameters["b1"]
    W2, b2 = parameters["W2"], parameters["b2"]
    W3, b3 = parameters["W3"], parameters["b3"]

    Z1 = np.dot(W1, X) + b1
    A1 = relu(Z1)
    # Step 1: random matrix the same shape as A1
    D1 = np.random.rand(A1.shape[0], A1.shape[1])
    # Step 2: turn it into a mask of 1s (keep) and 0s (drop)
    D1 = (D1 < keep_prob).astype(int)
    # Step 3: shut down the dropped neurons
    A1 = A1 * D1
    # Step 4: scale up survivors so the average signal is unchanged
    A1 = A1 / keep_prob

    Z2 = np.dot(W2, A1) + b2
    A2 = relu(Z2)
    D2 = np.random.rand(A2.shape[0], A2.shape[1]) # Step 1
    D2 = (D2 < keep_prob).astype(int) # Step 2
    A2 = A2 * D2 # Step 3
    A2 = A2 / keep_prob # Step 4

    Z3 = np.dot(W3, A2) + b3
    A3 = sigmoid(Z3)
```



```
cache = (Z1, D1, A1, W1, b1, Z2, D2, A2, W2, b2, Z3, A3, W3, b3)
return A3, cache
```

That fourth step — dividing by `keep_prob` — is subtle but important. If you drop, say, 14% of neurons, the layer's total signal shrinks. Dividing the survivors by `keep_prob` scales them back up so the average output stays the same as it would be with no dropout. This trick is called **inverted dropout**, and it is what lets you leave the test-time network untouched.

Just as with L2, dropout must also be applied during **backpropagation** — the same answer to the same question. Whatever neurons you switched off going forward, you must switch off the same ones coming back, using the very same masks (`D1`, `D2`) saved in the cache, and rescale by `keep_prob` again:

```
def backward_propagation_with_dropout(X, Y, cache, keep_prob):
    m = X.shape[1]
    (Z1, D1, A1, W1, b1, Z2, D2, A2, W2, b2, Z3, A3, W3, b3) = cache

    dZ3 = A3 - Y
    dW3 = 1./m * np.dot(dZ3, A2.T)
    db3 = 1./m * np.sum(dZ3, axis=1, keepdims=True)
    dA2 = np.dot(W3.T, dZ3)
    dA2 = dA2 * D2          # Step 1: reapply the SAME mask used in forward prop
    dA2 = dA2 / keep_prob    # Step 2: rescale, matching the forward-prop scaling

    dZ2 = np.multiply(dA2, np.int64(A2 > 0))
    dW2 = 1./m * np.dot(dZ2, A1.T)
    db2 = 1./m * np.sum(dZ2, axis=1, keepdims=True)
    dA1 = np.dot(W2.T, dZ2)
    dA1 = dA1 * D1          # Step 1: same mask again
    dA1 = dA1 / keep_prob    # Step 2: same rescaling

    dZ1 = np.multiply(dA1, np.int64(A1 > 0))
    dW1 = 1./m * np.dot(dZ1, X.T)
    db1 = 1./m * np.sum(dZ1, axis=1, keepdims=True)

    gradients = {"dZ3": dZ3, "dW3": dW3, "db3": db3, "dA2": dA2,
                  "dZ2": dZ2, "dW2": dW2, "db2": db2, "dA1": dA1,
                  "dZ1": dZ1, "dW1": dW1, "db1": db1}
    return gradients
```

Run with `keep_prob = 0.86`, the dropout model reached the best result of all: **95% test accuracy**, with a clean, smooth decision boundary and no sign of overfitting.

## WHAT TO REMEMBER ABOUT DROPOUT

Dropout is a regularization technique. Use it **only during training** — never at test time, when you want the full network and all its neurons making the prediction. Apply it during **both forward and backward propagation**, using the same masks each way. And always **divide by keep\_prob** during training so the activations keep the same expected value: if `keep_prob` is 0.5 you drop half the neurons on average, so dividing by 0.5 (the same as multiplying by 2) restores the original signal strength. A common beginner mistake is leaving dropout on at test time — don't.

## 6. Overfitting or Underfitting? How to Tell

This answers the assignment's final question, and it is a skill you will use constantly. The diagnosis comes from comparing two numbers: how well the model does on the **training set** versus how well it does on the **test (or development) set**.

### KEY CONCEPT: BIAS AND VARIANCE

**Underfitting** (called high bias) means the model is too simple to capture even the real pattern — it does poorly on the training set *and* on new data, with both scores low and close together. **Overfitting** (called high variance) means the model has learned the training set too well, including its noise — it scores high on training but noticeably lower on new data. The tell-tale sign of overfitting is the *gap* between a high training score and a lower test score.

The lecture notes make this concrete with cat-photo classification. A model with 1% training error but 11% development error is overfitting — it learned the training data but does not generalize, so it has high variance. A model with 15% training error and 16% development error is underfitting — it never fit the training data well in the first place, so it has high bias. And a model with 15% training error that worsens to 30% on the development set has both problems at once.

Your three football models trace this story exactly. The baseline scored 95% on training but only 91.5% on test — that four-point gap is overfitting, visible as the wiggly boundary chasing noise. L2 regularization closed the gap (94% train, 93% test), and dropout closed it further while pushing test accuracy to 95%. Each regularizer slightly lowered the training score, which is not a flaw but the whole point: it is the price of forcing the model to stop memorizing.

### THE QUICK DIAGNOSIS

Compare training performance with test (or development) performance. **Both poor and close together** → underfitting / high bias: the model is too weak, so make it bigger, train longer, or reduce regularization. **Training strong but test noticeably weaker** → overfitting / high variance: reach for regularization (L2 or dropout) or gather more data. **Both strong and close** → you have found the sweet spot.

## 7. The Three Models Side by Side

Same network, same data, same number of iterations — the only difference is the regularization applied. The results tell the whole story:

| Model                                    | Train Accuracy | Test Accuracy |
|------------------------------------------|----------------|---------------|
| 3-layer NN, no regularization (baseline) | 95%            | 91.5%         |
| 3-layer NN with L2 regularization        | 94%            | 93%           |
| 3-layer NN with dropout                  | 93%            | 95%           |

Read down the table and notice the pattern that surprises every beginner: regularization *hurts* training accuracy (95% → 94% → 93%) while *helping* test accuracy (91.5% → 93% → 95%). That is regularization working as designed. It limits the network's ability to overfit the training set, which costs a little there but pays off where it actually matters — on data the model has never seen.

## 8. Putting It All Together — How You Use Regularization

So, how do you use regularization, and what are its components? The workflow you just practiced is the answer in miniature. First, build and train a **baseline model** with no regularization, and check it for overfitting by comparing its training and test accuracy and inspecting its decision boundary. If the training score is high but the test score lags — the high-variance signature — reach for a regularizer.

The two components in your toolkit work through different parts of the same training loop. **L2 regularization** has two moving parts: a penalty added to the *cost*, and a matching extra term added to the weight gradients in *backpropagation*; together they drive the weights toward smaller values. **Dropout** also has two moving parts: a random mask applied in *forward propagation* (with rescaling by `keep_prob`), and that same mask reapplied in *backpropagation*; together they stop the network relying on any single neuron. In both cases the cost tells the network how wrong it is, and backpropagation carries the regularization pressure into every weight update — which is why both techniques must touch both halves of the loop.

### THE MAIN TAKEAWAYS

Regularization helps you reduce overfitting. L2 regularization drives your weights toward lower values. L2 regularization and dropout are two very effective regularization techniques. Always begin from a baseline, diagnose overfitting by the gap between training and test accuracy, and remember that a small drop in training accuracy is the expected, healthy cost of a model that finally generalizes.

The deepest lesson is the one the football table makes vivid. A more powerful model is not automatically a better one; left unchecked, all that capacity goes into memorizing noise. Regularization is how you spend a network's power wisely — holding it back just enough that it learns the real pattern instead of the accidents in the data. That balance, between fitting and generalizing, is one of the central crafts of deep learning, and you have now practiced both of its most important tools.